PROBLÈME · ANNALES SPÉ · 45 MIN

Mots de Dyck et algorithme de Heap

`let rec dyck = let f (x,y) = "a" ^ x ^ "b" ^ y in Cons ("", fun () -> map f (product dyck dyck))` ; D est l'ensemble des mots de ce flot.

1. Donner une grammaire non contextuelle G avec $L(G) = D$.
2. Énoncer une propriété de G qui assure que `dyck` n'énumère jamais deux fois le même mot, et la démontrer.

Algorithme de Heap : Heap(k, t) : si $k = 1$, afficher t ; sinon, pour $i = 0$ à $k - 1$: exécuter Heap($k - 1$, t), puis si k est pair échanger $t[i]$ et $t[k - 1]$, sinon échanger $t[0]$ et $t[k - 1]$.

1. Nombre $A(k)$ d'affichages lors de Heap(k, t).
2. Montrer : après Heap(k, t), si k est impair, t est inchangé ; si k est pair, les k premières cases ont subi une permutation circulaire vers la droite.
3. Montrer que Heap(k, t) affiche chaque permutation des k premières cases (les autres inchangées).
4. Écrire `pivot (k:int) (i:int) (t:'a array) : unit` (lignes 6 à 9).
5. Écrire `permstream_of_array (t:'a array) : 'a array stream`, flot fini des permutations dans l'ordre de l'algorithme de Heap, économe en mémoire.